

```

DECLARE
    sub1 NUMBER := 85;
    sub2 NUMBER := 78;
    sub3 NUMBER := 92;
    sub4 NUMBER := 88;

    total NUMBER;
    avg NUMBER;
    grade VARCHAR2(10);
BEGIN

    total := sub1 + sub2 + sub3 + sub4;

    avg := total / 4;

    IF avg >= 90 THEN
        grade := 'A+';
    ELSIF avg >= 75 THEN
        grade := 'A';
    ELSIF avg >= 60 THEN
        grade := 'B';
    ELSIF avg >= 50 THEN
        grade := 'C';
    ELSE
        grade := 'F';
    END IF;

    DBMS_OUTPUT.PUT_LINE('Total Marks = ' || total);
    DBMS_OUTPUT.PUT_LINE('Average Marks = ' || avg);
    DBMS_OUTPUT.PUT_LINE('Grade = ' || grade);
END;
/

```

Output:

Total Marks = 343

Average Marks = 85.75

Grade = A

```

DECLARE
    a NUMBER := 25;
    b NUMBER := 40;
    c NUMBER := 30;

    largest NUMBER;
BEGIN

    IF a >= b AND a >= c THEN
        largest := a;
    ELSIF b >= a AND b >= c THEN
        largest := b;
    ELSE
        largest := c;
    END IF;

    DBMS_OUTPUT.PUT_LINE('Largest Number = ' || largest);
END;
/

```

Output:

Largest Number = 40

```

DECLARE
    n NUMBER := 10; -- number of terms
    first NUMBER := 0;
    second NUMBER := 1;
    next NUMBER;
    i NUMBER;
BEGIN
    DBMS_OUTPUT.PUT_LINE('Fibonacci Series:');

    -- Print first two terms
    DBMS_OUTPUT.PUT_LINE(first);
    DBMS_OUTPUT.PUT_LINE(second);

    -- Generate remaining terms
    FOR i IN 3..n LOOP
        next := first + second;
        DBMS_OUTPUT.PUT_LINE(next);

        first := second;
        second := next;
    END LOOP;
END;
/

```

Output:

Fibonacci Series:

```

0
1
1
2
3
5
8
13
21
34

```

```

CREATE OR REPLACE PROCEDURE find_max (
    a IN NUMBER,
    b IN NUMBER,
    result OUT NUMBER
)
IS
BEGIN
    IF a > b THEN
        result := a;
    ELSE
        result := b;
    END IF;
END;
/

DECLARE
    res NUMBER;
BEGIN
    find_max(15, 30, res);
    DBMS_OUTPUT.PUT_LINE('Maximum = ' || res);
END;
/

```

Output:

Maximum = 30

```
CREATE OR REPLACE PROCEDURE find_square (  
    num IN NUMBER,  
    sq OUT NUMBER  
)  
IS  
BEGIN  
    sq := num * num;  
END;  
/  
  
-- Execution Block  
DECLARE  
    result NUMBER;  
BEGIN  
    find_square(7, result);  
    DBMS_OUTPUT.PUT_LINE('Square = ' || result);  
END;  
/
```

Output:

Square = 49

```
SET SERVEROUTPUT ON;
```

```
CREATE TABLE emp (  
    eid NUMBER,  
    ename VARCHAR2(50),  
    salary NUMBER,  
    dept VARCHAR2(50)  
);
```

```
CREATE OR REPLACE TRIGGER emp_insert_trigger  
AFTER INSERT  
ON emp  
FOR EACH ROW  
BEGIN  
    DBMS_OUTPUT.PUT_LINE('New Employee Added');  
    DBMS_OUTPUT.PUT_LINE('Employee ID: ' || :NEW.eid);  
    DBMS_OUTPUT.PUT_LINE('Employee Name: ' || :NEW.ename);  
END;  
/
```

```
CREATE OR REPLACE TRIGGER emp_delete_trigger  
AFTER DELETE  
ON emp  
FOR EACH ROW  
BEGIN  
    DBMS_OUTPUT.PUT_LINE('Employee Deleted');  
    DBMS_OUTPUT.PUT_LINE('Employee ID: ' || :OLD.eid);  
    DBMS_OUTPUT.PUT_LINE('Employee Name: ' || :OLD.ename);  
END;  
/
```

```
CREATE OR REPLACE TRIGGER emp_update_trigger  
AFTER UPDATE  
ON emp  
FOR EACH ROW  
BEGIN  
    DBMS_OUTPUT.PUT_LINE('Employee Updated');  
    DBMS_OUTPUT.PUT_LINE('Old Salary: ' || :OLD.salary);  
    DBMS_OUTPUT.PUT_LINE('New Salary: ' || :NEW.salary);  
END;  
/
```

```
INSERT INTO emp VALUES (101, 'Ravi', 40000, 'IT');
```

```
UPDATE emp SET salary = 50000 WHERE eid = 101;
```

```
DELETE FROM emp WHERE eid = 101;
```

New Employee Added

Employee ID: 101

Employee Name: Ravi

Employee Updated

Old Salary: 40000

New Salary: 50000

Employee Deleted

Employee ID: 101

Employee Name: Ravi